Intro to Calcul Quebec!

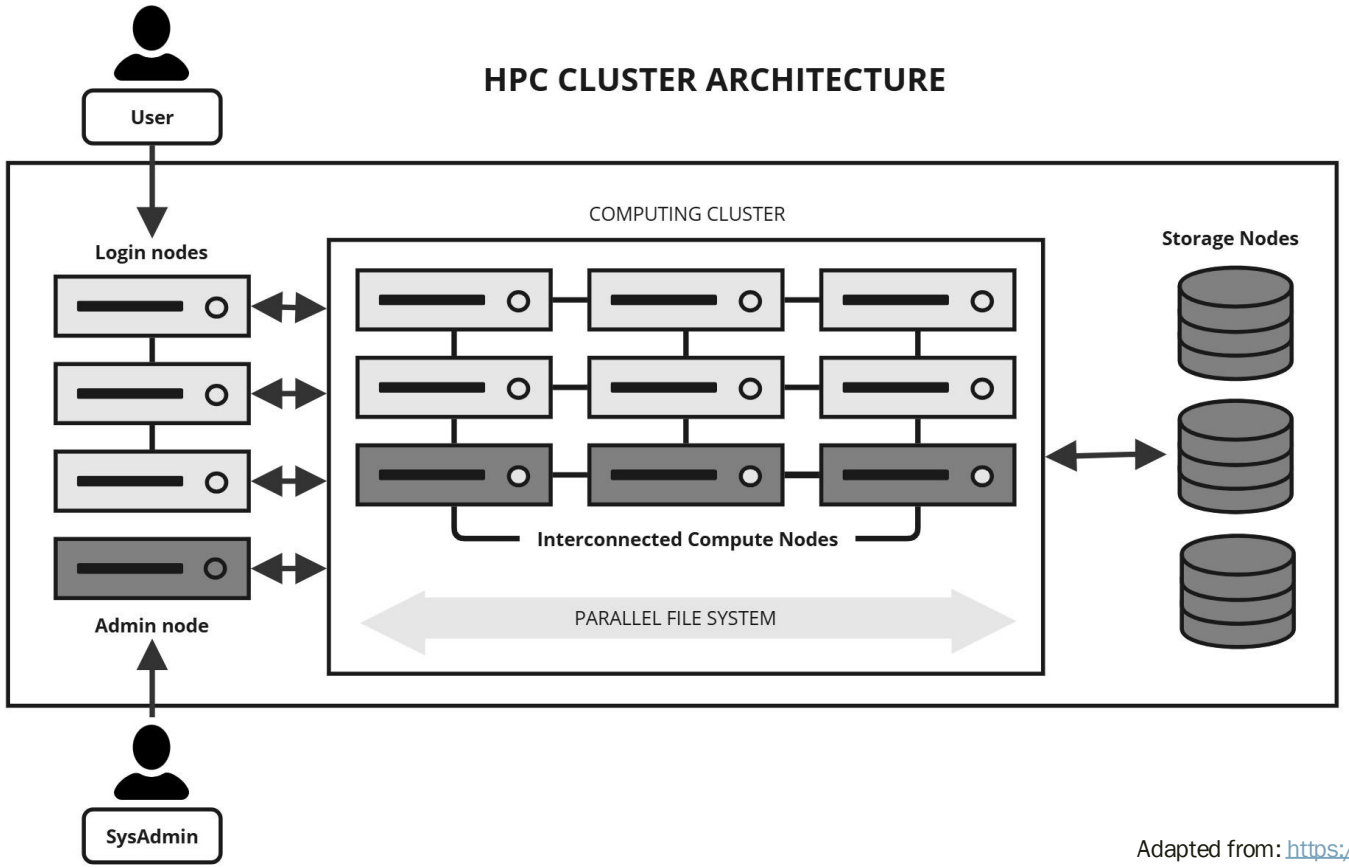
03/27/2026

Overview

HPC Systems

A traditional computing system will solve problems by running tasks sequentially on the same processor.

Parallelizing it is much more efficient, wherein we can instead run these tasks simultaneously via clusters!



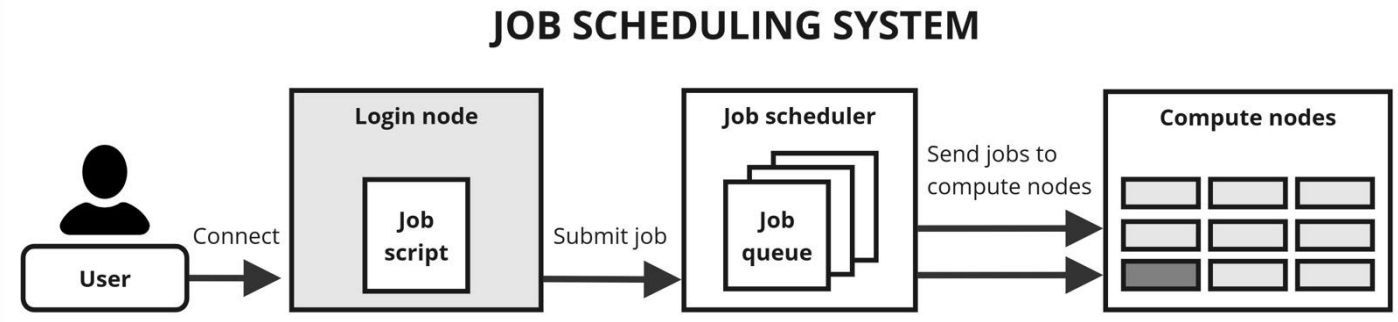
Adapted from: https://arcdocs.leeds.ac.uk/aire/system/hpc_architecture.html

Nodes/Clusters

A cluster consists of a group of nodes that work together to run computational jobs.

Login nodes: do not run jobs, but we can SSH to it to edit files, dispatch runs, and manage data.

Compute nodes: do run jobs, consisting of many cores (processing units) that can be executed in parallel.



The clusters available are Fir, Narval, Nibi, Rorqual, and Trillium.

Fir and Narval are general purpose clusters, while Rorqyal and Nibi are heterogeneous clusters. Trillium is a specially designed cluster for large parallel jobs (>1000 cores).

Note: both Nibi and Fir's compute nodes have access to the internet.

Source: https://arcdocs.leeds.ac.uk/aire/system/hpc_architecture.html

Starting...

Account Setup

1. Create an account here: https://docs.alliancecan.ca/wiki/Apply_for_a_CCDB_account
   - *If you don't already have one, Seb must approve it!*

2. Multifactor authentication: https://docs.alliancecan.ca/wiki/Multifactor_authentication
   - *Via Duo mobile app is the easiest if your phone/laptop supports it!*

3. SSH configuration: https://docs.alliancecan.ca/wiki/SSH_Keys
   - *Once you have an SSH key generated, you can paste it into this form here: https://ccdb.alliancecan.ca/ssh_authorized_keys so it can be used in any of the clusters.*
   - *Update .ssh/config file accordingly:*

```
Host *
  ServerAliveInterval 30

Host rorqual narval nibi fir
    HostName %h.alliancecan.ca
    User chans
    IdentityFile ~/.ssh/cq_key
    AddKeysToAgent yes
    UseKeychain yes
```

Optional: can use ssh-agent in Linux (https://docs.alliancecan.ca/wiki/Using_SSH_keys_in_Linux) so you don't have to input password each time you login!

```
[chans@rorqual2 ~]$ diskusage_report
                             Description                Space          # of files
                       /home (user chans)      12GB/   50GB          36K/ 500K
                    /scratch (user chans)      19GB/   20TB          37 /1000K
/project (project def-lemieux1)               146MB/1000GB          10 / 500K
```

File Systems

~/links/

home/: 50GB, 500K files per user

- Source code, small parameter files, job submission scripts
- Mediocre performance for read/write on large files

scratch/: 20TB, 1M files per user

- ***Not backed up!!! files older than 60d are purged.
- Temporary files, checkpointing, job outputs
- Best performance for read/write operations on large (>100MB) files

projects/: 1TB, 500K files per group

- Good for sharing data within lab (linked to PI's account, not individual user)
- Data should remain fairly static; frequent changes is hard on backup system

File Transfers

Git

You likely already use GitHub with your current code locally or with IRIC's clusters. To load in your existing repo's contents into your newly created folder, do:

[name@server ~]$ git clone git://github.com/git/git.git

Globus

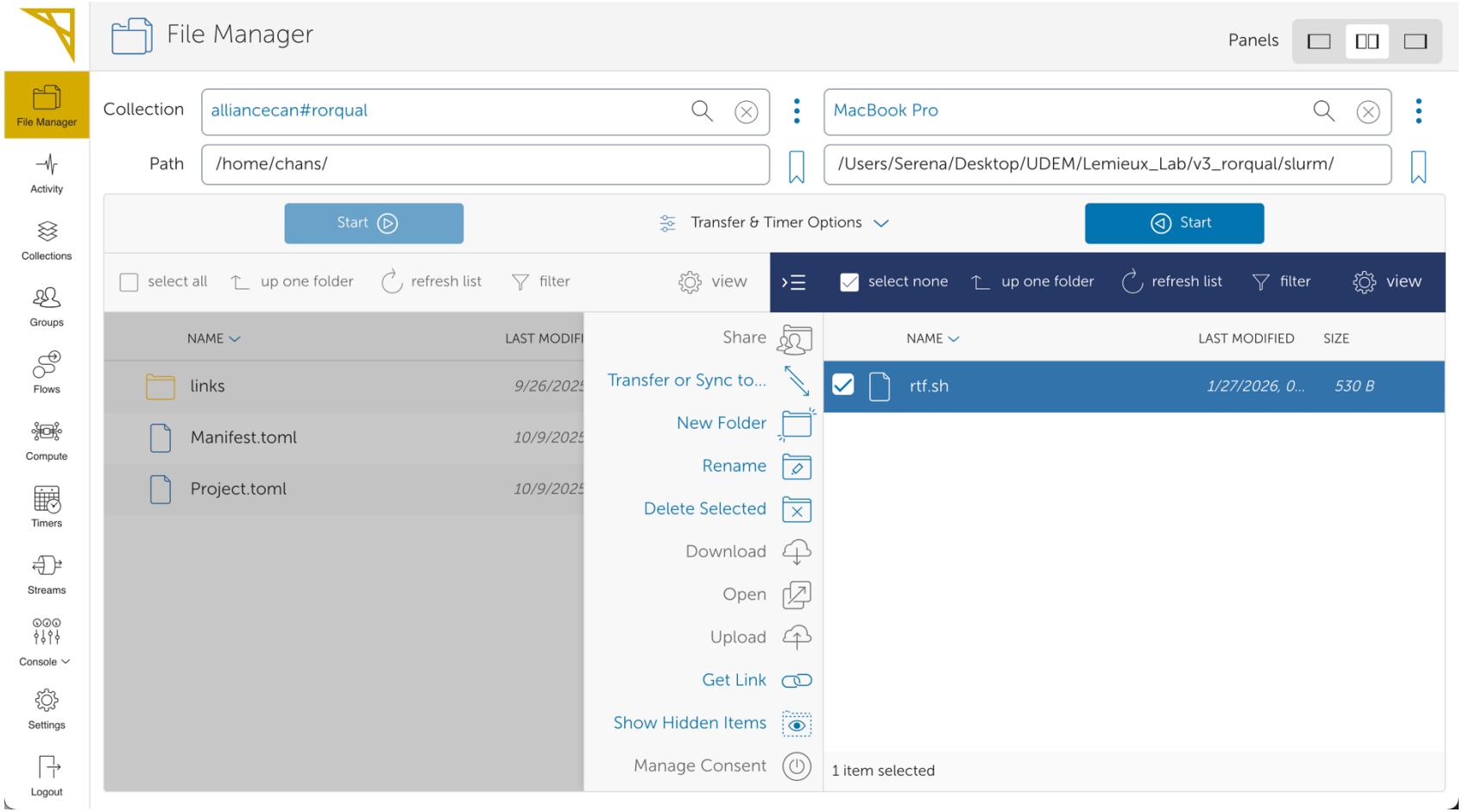
CQ provides Globus: https://globus.alliancecan.ca/ for transferring larger files to and from your own computer. Fast and easy to navigate if you don't already know how to use sftp, scp, or rsync.

SFTP, SCP, rsync

See info here: https://docs.alliancecan.ca/wiki/Transferring_data

Globus Example

You must have Globus Connect Personal installed on wherever your files are!



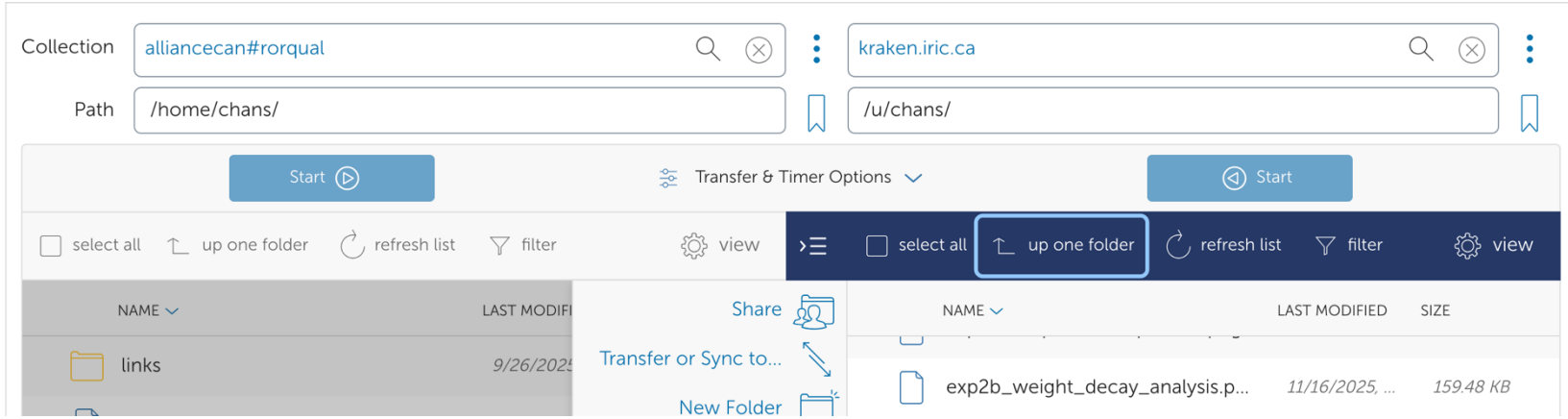
Globus Example

Can do via CLI on your workstation: https://docs.globus.org/globus-connect-personal/install/linux/

Once installed, you can run via:

```
[chans@kraken globusconnectpersonal-3.2.7]$ ./globusconnectpersonal -start &
[1] 565355
[chans@kraken globusconnectpersonal-3.2.7]$ ./globusconnectpersonal -status
Globus Online:   connected
Transfer Status: idle
```

Then we can connect to IRIC!



Connecting...

Login Node

#!# Do not run or debug code when connected through login node

Terminal

[name@server ~]$ ssh username@hostname.alliancecan.ca

VSCode

You must update your JSON settings both on the local side and remote side (https://docs.alliancecan.ca/wiki/Visual_Studio_Code#Local_configuration).

However, it is very discouraged to connect your VSCode to the login node (ex. it's banned on Fir), so you don't have to do this if you won't use VSCode without a compute node.

Compute Node – Serial Jobs

A serial job is the simplest type of job; it only requests one core. To submit this, we use call sbatch on our .sh script.

Example:

```
#!/bin/bash
#SBATCH --time=00:05:00
#SBATCH --account=def-lemieux1

#SBATCH --job-name=serial
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G

#SBATCH --output=out/serial_%A.out
#SBATCH --error=out/serial_%A.out

#SBATCH --mail-user=serenaktc@gmail.com
#SBATCH --mail-type=ALL

echo "hello!!!"
```

Note: can add this into your .bashrc if you're using the same account often:

```
export SLURM_ACCOUNT=def-lemieux1
export SBATCH_ACCOUNT=$SLURM_ACCOUNT
export SALLOC_ACCOUNT=$SLURM_ACCOUNT
```

Compute Node – Array Jobs

An array job submits a certain set of jobs with a singular command. To submit this, we use call sbatch on our .sh script.

Example:

```
#!/bin/bash
#SBATCH --time=00:05:00
#SBATCH --account=def-lemieux1
#SBATCH --array=1-10

#SBATCH --job-name=array
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=1G

#SBATCH --output=out/array_%A_%a.out
#SBATCH --error=out/array_%A_%a.out

#SBATCH --mail-user=serenaktc@gmail.com
#SBATCH --mail-type=ALL

echo "hello!!! id: $SLURM_ARRAY_TASK_ID"
```

Note: Per each full GPU requested, it is recommended to have at most:

- Fir, Narval: 12 CPU cores
- Nibi: 14 CPU cores
- Rorqual: 16 CPU cores

Compute Node – Interactive

Update your ssh config file with the following:

```
Host nc* ng* nl*
    ProxyJump narval
    User chans
```

```
Host rc* rg* rl*
    ProxyJump rorqual
    User chans
```

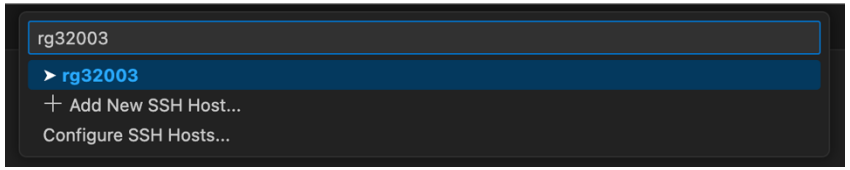
Then we can connect via salloc in the terminal (w/ at least 2000M of memory):

[name@server ~]$ salloc - -time=00:15:00, - -mem-per-cpu=3G, - -ntasks=1, - -account=def-someuser

Once allocated, we get something like this:

```
salloc: Pending job allocation 9096218
salloc: job 9096218 queued and waiting for resources
salloc: job 9096218 has been allocated resources
salloc: Granted job allocation 9096218
salloc: Nodes rg32003 are ready for job
```

Type this into VSCode's Remote SSH here:



If you need to use SLURM_* environment variables in VSCode, save them all in a source file, then use the VSCode terminal to source the variables.sh file.

Note: there are nodes dedicated for jobs 3hrs or less, if you request more than this amount you will likely have to wait longer periods of time.

Julia Specifics

Compilation

Since we don't have internet when connected to the compute node, it's necessary to prepare your packages beforehand. Additionally, you must load the pre-installed modules so Julia knows where to find them!

Login node:

$ module load julia cuda

$ julia --project=. -e ' ENV["JULIA_PKG_PRECOMPILE_AUTO"]=0; using Pkg; Pkg.instantiate();'

$ salloc…

Compute node:

$ module load julia cuda

$ run your code…

Note: on Narval, Julia crashes sometimes when installing pkgs in /home during precompile!

Monitoring...

RGUs

Each GPU in CQ is ranked in terms of allocation weight using Reference GPU Units (RGUs). This is determined by its performance on FP16 (40%), FP32 (40%), and memory (20%).

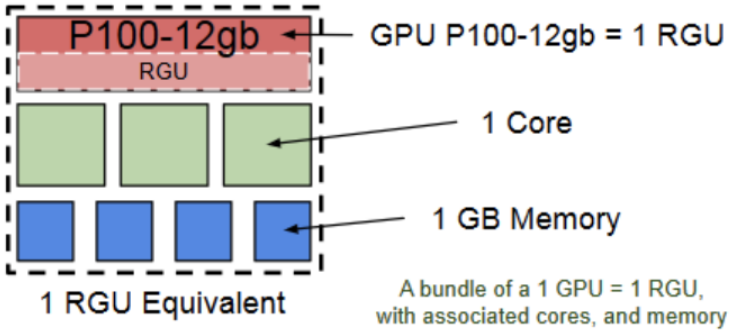
	FP32 score	FP16 score	Memory score	Combined score
Coefficient:	**1.6**	**1.6**	**0.8**	**(RGU)**
H100-80gb	3.44	3.17	2.0	12.2
A100-80gb	1.00	1.00	2.0	4.8
A100-40gb	**1.00**	**1.00**	**1.0**	**4.0**

To determine which GPU should be used, you can look at whether your applications are primarily doing FP32 or FP16 operations.

Usage

Consequently, this also affects resource usage.

Research groups are charged by either cores, RGUs, or memory (whichever one is requested more of) per bundle.



Generally, it's good to just follow the recommended in order to maximize computation and minimize resources.

Cluster	Model or instance	RGU per GPU	Bundle per GPU	Recommended per GPU
Fir	**H100-80gb**	**12.2**	**12 cores, 288 GB**	**12 cores, 280 GB**
	H100-1g.10gb	1.74	1.7 cores, 41 GB	1 core, 35 GB
	H100-2g.20gb	3.48	3.4 cores, 82 GB	3 cores, 70 GB
	H100-3g.40gb	6.1	6 cores, 144 GB	6 cores, 140 GB
Narval	**A100-40gb**	**4.0**	**12 cores, 124.5 GB**	**12 cores, 124 GB**
	A100-1g.5gb	0.57	1.7 cores, 17.7 GB	1 core, 15 GB
	A100-2g.10gb	1.14	3.4 cores, 35.4 GB	3 cores, 31 GB
	A100-3g.20gb	2.0	6.0 cores, 62.2 GB	6 cores, 62 GB
	A100-4g.20gb	2.3	6.9 cores, 71.5 GB	6 cores, 62 GB
Nibi	**H100-80gb**	**12.2**	**14 cores, 250 GB**	**14 cores, 250 GB**
	H100-1g.10gb	1.74	2 cores, 35.7 GB	2 cores, 31 GB
	H100-2g.20gb	3.48	4 cores, 71.4 GB	4 cores, 62 GB
	H100-3g.40gb	6.1	7 cores, 125 GB	6 cores, 124 GB
Rorqual	**H100-80gb**	**12.2**	**16 cores, 124.5 GB**	**16 cores, 124 GB**
	H100-1g.10gb	1.74	2.3 cores, 17.7 GB	2 cores, 15 GB
	H100-2g.20gb	3.48	4.5 cores, 35.4 GB	4 cores, 31 GB
	H100-3g.40gb	6.1	8 cores, 62.2 GB	8 cores, 62 GB
Trillium	**H100-80gb**	**12.2**	**24 cores, 188 GB**	**24 cores, 188 GB**

Checkpointing

When a computation requires a long time to complete (>7days), or your process has the chance of being killed at random (ex. last week's discussion…), it should be able to save a checkpoint file, then restart + continue from that saved state.

Two methods:

- Slurm job arrays
  - *Via --array=1-100%10*
  - *Submit a collection of identical jobs wherein only 1 will run at any given time w/ the last checkpoint used for the next job*
- Resubmitting from the end of the job script
  - *Run calculations in chunks; once 1 chunks has finished but the job's runtime hasn't finished, check if end of calculation has been reached*
  - *If not, it runs another sbatch of itself to continue working*

Customizing w/ Nic!